GhostHouses

Stage B Presentation
2025-2026-Spring Semester
02340312 - Yearly Project in Software Eng.-Stage B

GhostHouses Team

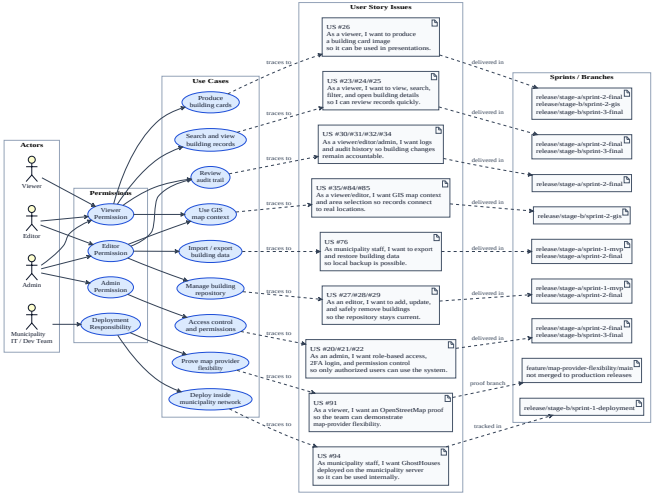
Technion, Faculty of Computer Science

Agenda

1. User Stories to Use Cases to Issues to Sprints
2. Architecture
3. Customer-Side Deployment

User Stories to Use Cases to Issues to Sprints

Traceability UML



User Stories to Use Cases to Issues to Sprints

- We grouped the backlog by actors, permission levels, and real user-facing use cases.
- Each use case maps to readable GitHub User Story issues, not only issue numbers.
- The diagram keeps implementation sub-issues out of the slide so the presentation stays readable.
- Release branches capture sprint checkpoints: Stage A MVP, Stage A final, Stage B deployment, Stage B GIS, and the final handoff state.

Branch Management / Issue Management

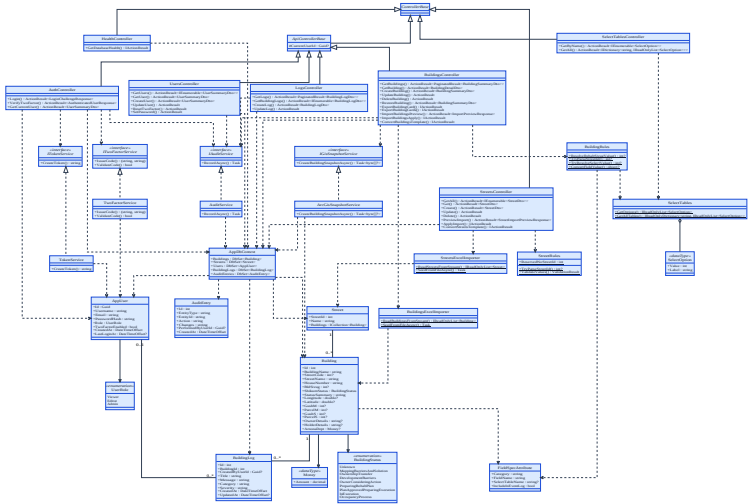
- **Branch management:** branches are organized around `develop`, feature branches, and release checkpoints.
- **Branch management:** feature work follows `develop` → `feature/<feature>/main` → `feature/<feature>/#<issue>-<slug>` → `feature/<feature>/main` → `develop`.
- **Issue management:** User Story issues describe user value and acceptance criteria.
- **Issue management:** implementation issues are linked to one parent User Story and one development branch.
- **Issue management:** closed implementation issues include a short completion comment with a `Time spent` line.

Automated Tests / Readme.md / License

- **Automated tests:** GitHub Actions runs backend build/tests, frontend build, and Docker Compose build readiness.
- **Automated tests:** Issue Guard checks labels, milestone, project status, parent links, branches, and User Story acceptance criteria.
- **Readme.md:** the README is structured as a handoff guide with a clear Project Delivery section.
- **License:** the repository includes a Creative Commons Attribution 4.0 International license.

Architecture

UML Class Diagram



Good Use of Abstract Classes and Design Patterns

- Abstract boundaries: service contracts for JWT, OTP, audit logging, and GIS snapshots.
- Concrete commitments: ASP.NET Core backend, React frontend, PostgreSQL database, ArcGIS GIS provider, and Docker Compose deployment.
- The external-risk areas stay behind interfaces, so OTP delivery or GIS provider changes do not require rewriting controllers or the domain model.
- Stable runtime choices are concrete because they define how the municipality receives and operates the system.

Good Use of Abstract Classes and Design Patterns

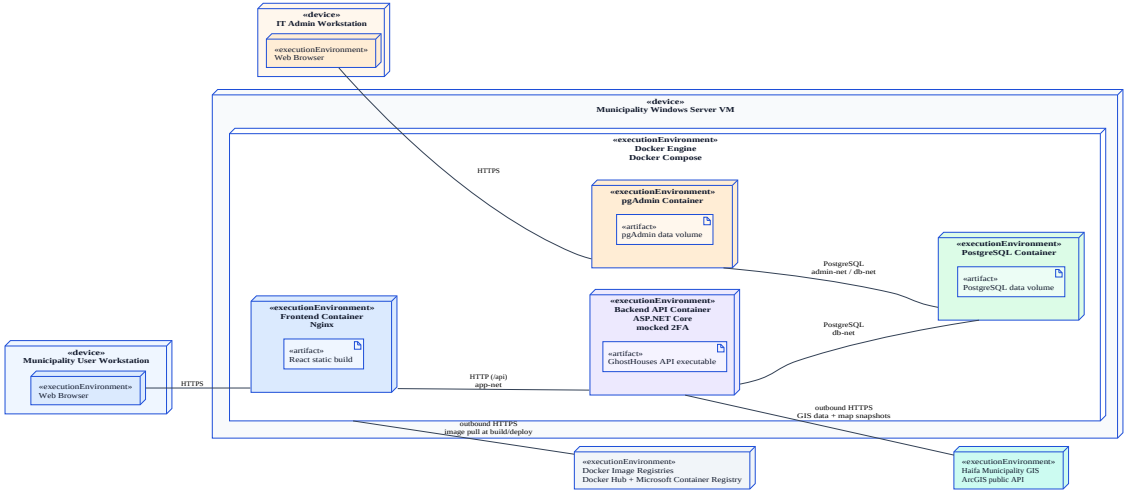
- `ApiControllerBase` centralizes authenticated API behavior.
- `BuildingRules` and `StreetRules` keep manual editing and Excel import aligned to one source of truth.
- `AppDbContext` is the persistence boundary for PostgreSQL entities.
- High cohesion: controllers, services, validation rules, import helpers, and models each have a clear responsibility.
- Low coupling: controllers depend on service contracts and stable boundaries instead of direct external systems.

Robustness to Next Pivot / API Vendor Change

- **Future proof:** `IGisSnapshotService` protects building-card export from a future GIS vendor change.
- The delivered system uses Haifa Municipality ArcGIS because it is the client-approved provider.
- A separate OpenStreetMap proof branch demonstrates that the map provider can be replaced without changing the production delivery.
- OTP is mocked for handoff, but the flow depends on `ITwoFactorService`, so a real SMS/email provider can replace it later.
- Centralized business rules make future municipality field or validation changes safer.

Customer-Side Deployment

Deployment UML



Customer-Side Deployment

- The municipality environment is a Windows Server VM inside the private municipality network.
- The Docker Compose stack was run successfully with frontend, backend, PostgreSQL, and pgAdmin.
- Municipality users access only the frontend through HTTPS 443.
- IT/database administration uses pgAdmin through HTTPS 8443, we walked through tables and users with the municipality dev team.
- Backend and database stay internal to Docker networks and are not exposed directly.

Customer-Side Deployment Woes and Mitigation

- We verified the normal run command from `project/`: `docker compose up -d --build`.
- We verified the clean reset command: `docker compose down -v && docker compose up -d --build`.
- We explained production `.env` values, real certificates, domain/DNS, port forwarding, and restricted pgAdmin access.
- Remaining production infrastructure decisions, including network routing and admin VLAN access, are municipality-owned.

Exhausting Communication With the Customer

- **Exhausting communication with the customer toward possible deployment, in writing:** deployment communication is documented with the client side, municipality dev team, IT/security contacts, and supervisors.
- The latest handoff covered run/reset commands, pgAdmin table inspection, `.env` values, certificates, domain/DNS, port forwarding, and VLAN-limited admin access.
- We explained the mocked 2FA replacement point and the Swagger toggle for temporary API/security review.
- The deployment handoff was completed after the walkthrough.

Thank You

GhostHouses Team